论文解读 · arXiv 2610.35850

OSWAM：用「想象 token」把世界动作模型的动作生成压到一步

OSWAM: An Effective World Action Model with One-Step Action Generation

Guowen Zhang*, Liyi Chen*, Xu Hu, Pengfei Wang, Jingfan Chen, Chenhang He, Lei Zhang †
香港理工大学 Visual Computing Lab · 首次提交 2026-10-26

自动驾驶 · NAVSIM / nuScenes

双臂操作 · RoboTwin 2.0

真机 · PiPER 机械臂

一句话总结

用结构化先验替代纯噪声起点：动作去噪 10 步 → 1 步，精度基本不掉

OSWAM 在 Video DiT 里加入少量「想象 token」（AIT / SIT），一次前向就得到结构化动作先验，代替高斯噪声作为 Action DiT 的起点；再用能看到未来帧的教师做特征蒸馏，使动作只需 **一步去噪**。

10 → 1

动作去噪步数

直接策略式 WAM （Fast-WAM / SimWAM）需约 10 步

≈3×

端到端提速

NAVSIM 136 ms vs SimWAM 410 ms（H20）

90.3 / 90.0

PDMS / RoboTwin 成功率

SimWAM 90.3； Fast-WAM 91.83（− 1.83）

40.8 ms

编译后延迟

RoboTwin， RTX 5090，torch.compile + CUDA Graph

定位：沿用 Fast-WAM 的直接策略式框架，只解决剩下的“动作多步去噪”这一瓶颈。

三类世界动作模型：直接策略式已去掉视频生成，但动作仍要约 10 步去噪

先想象再执行

逆动力学 WAM

$p(a|o,l) = \int p(v|o,l) \cdot p(a|o,l,v)\, dv$

推理时先生成未来视频，再反推动作

LingBot-VA **7659 ms**

DriveWAM **1032 ms**

Joint-WAM

视频 - 动作联合生成

$p(a, v \mid o, l)$

视频与动作在同一生成过程中联合去噪

Motus **2367 ms**

直接策略式 WAM

Fast-WAM / SimWAM

$p(a|o,l) = p(a \mid z(o))$

视频预测仅作训练信号，推理只编码首帧

Fast-WAM **397 ms**

SimWAM **410 ms**

剩余瓶颈：直接策略式仍要约 10 步 flow-matching 动作去噪（K≈10）→ OSWAM 的目标：**K = 1**

出处： Sec. 3.1， Table 1/2 （延迟为 H20 实测）

问题

直接把步数砍到一步会崩：随机噪声里没有动作结构

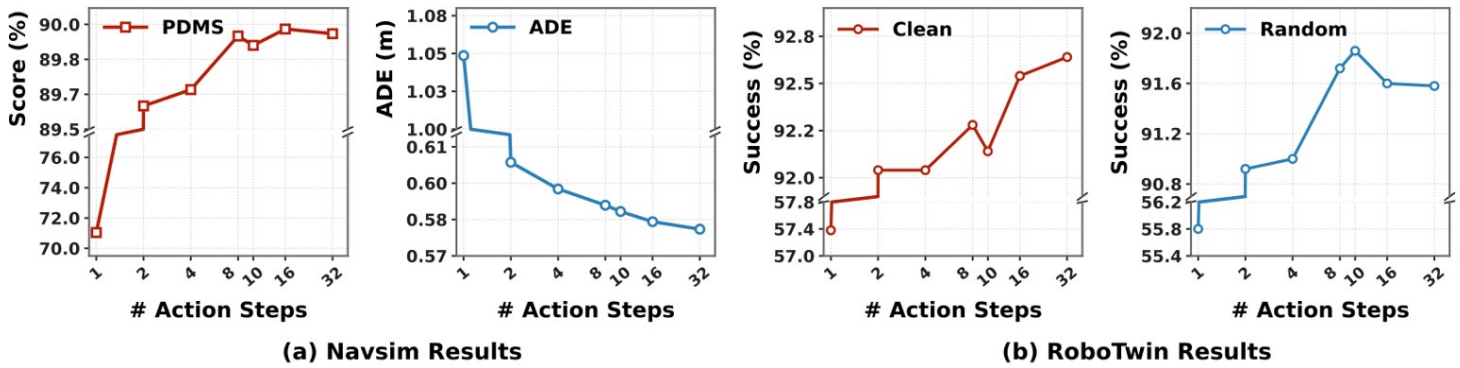


Fig. 2： Fast-WAM 在不同动作去噪步数下的表现（左： NAVSIM PDMS / ADE；右： RoboTwin Clean / Random ）

↓ ≈20%

NAVSIM： 一步时 PDMS 下降

↓ ≈40%

RoboTwin： 一步时成功率下降

附录 Table A.4： 基线的一步结果

方法（1 步）	指标	结果
SimWAM	NAVSIM PDMS	68.9
DriveVA	NAVSIM PDMS	13.2
Fast-WAM	RoboTwin 平均	56.59%

根因假设：一步更新必须从纯噪声一次恢复连贯轨迹；步数蒸馏 / rectified flow 仍从高斯噪声出发 → **需要结构化的初始化**

换掉起点：从“纯噪声 + 10 步积分”变为“结构化先验 + 1 步修正”

直接策略式 WAM

Fast-WAM / SimWAM



动作 GPU 时间 67.61 ms （Fast-WAM， RoboTwin， RTX 5090）

OSWAM

本文

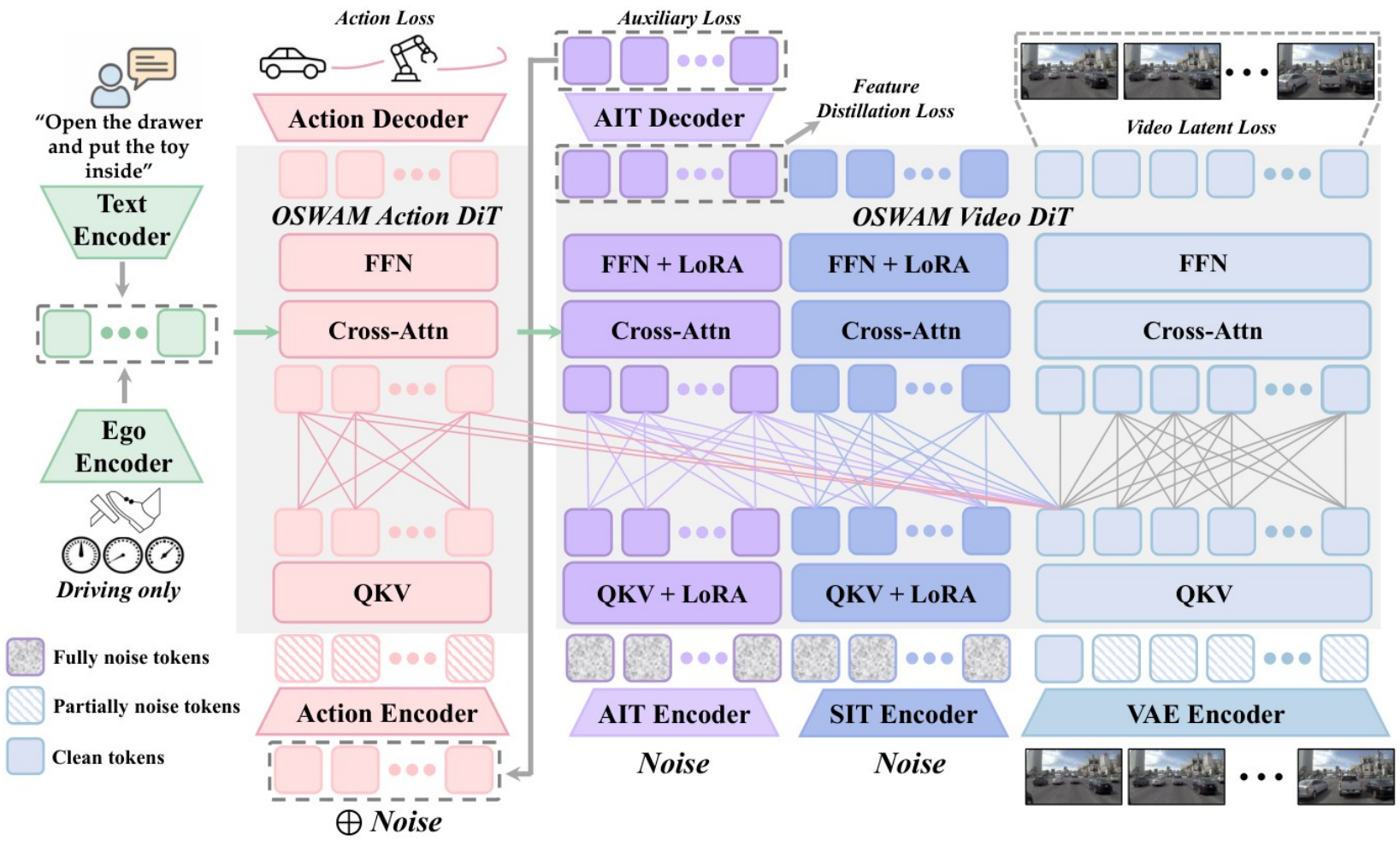


动作 GPU 时间 6.78 ms； Video DiT 仅增加 5.48 ms

一步更新（式 4）： $\hat{a} = a_t - t \cdot v_\theta(a_t, t)$

纯噪声起点 t = 1：一次更新要跨过整条去噪轨迹，容易得到粗糙动作； OSWAM 从 t < 1 的结构化中间态出发， Action DiT 只需修正残差。

Video DiT 一次前向，同时产出首帧特征和动作先验



Action DiT （粉色）

以 AIT 先验为起点，只做 1 次去噪；由 Action Loss 监督

AIT / SIT （紫 / 蓝）

噪声初始化的想象 token ， LoRA 适配；AIT 有辅助损失和蒸馏损失， SIT 没有直接损失

视频 token （浅蓝）

首帧干净，未来帧加噪；未来帧仅用于训练时的 Video Latent Loss

条件输入

T5 文本 + Ego 编码器（仅驾驶）经 cross-attention 注入； Video 与 Action 共享 MoT 注意力

AIT 先回归一个粗动作，重加噪后交给 Action DiT 一步修正

1

AIT：动作想象 token

编码动作噪声 ε， t_max = 1000；注意首帧 + SIT + AIT → 辅助解码器得到先验 ā （L_AIT 监督）

2

SIT：场景想象 token

独立噪声初始化，只注意首帧 + SIT；充当场景上下文槽位，无直接损失，经 AIT 注意力间接学习

3

重加噪

截断梯度 sg(ā)，用同一个 ε 混合到 σ_r = 0.5，保留先验结构的同时留出修正空间

4

Action DiT 一步细化

以首帧特征和任务为条件预测 v_act；目标除以 σ_r，一步 Euler 即可落到真值

关键公式

AIT 先验

$$\bar{a} = \varepsilon - v_{AIT}$$

重加噪（式 5）

$$a_{init} = (1-\sigma_r)\cdot sg(\bar{a}) + \sigma_r \cdot \varepsilon$$

一步去噪（式 6）

$$\hat{a} = a_{init} - \sigma_r \cdot v_{act}$$

训练目标（式 11）

$$v_{act} \rightarrow (a_{init} - a_{gt}) / \sigma_r$$

总损失（式 12）

$$L = L_{vid} + L_{AIT} + \lambda \cdot L_{dist} + L_{act}$$

t_max = 1000， τ_r = 500 （σ_r = 0.5），λ_dist = 10

结构化掩码保证只用首帧推理；角色感知 RoPE 区分三类 token

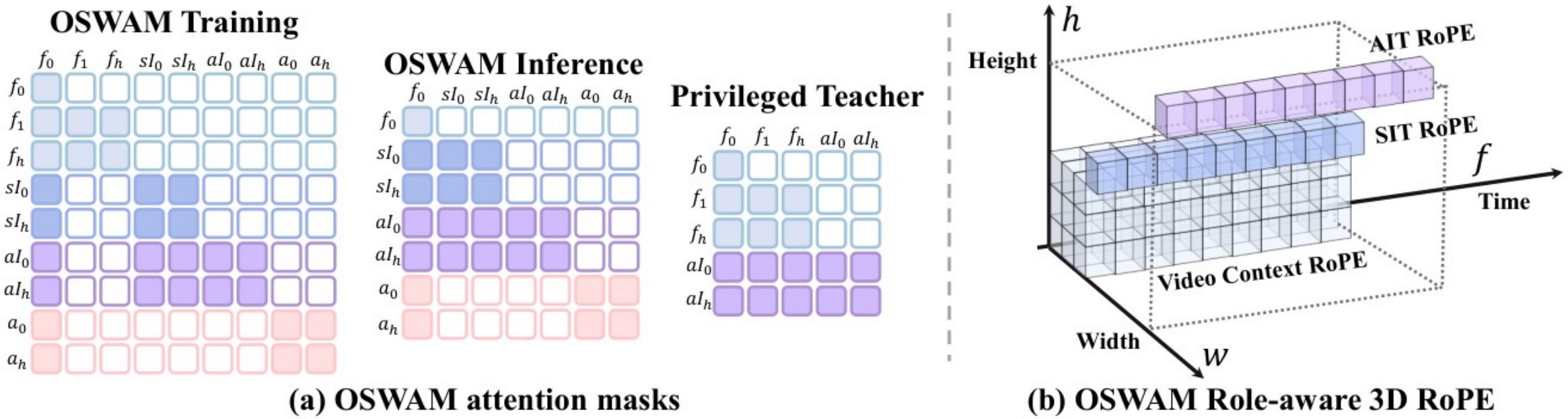


Fig. 4： (a) OSWAM 训练 / 推理及特权教师的注意力掩码； (b) Role-aware 3D RoPE （ sI = SIT， aI = AIT ）

注意力掩码

SIT → 首帧 + SIT； AIT → 首帧 + SIT + AIT；动作 → 首帧 + 动作

SIT / AIT / 动作 token 都不依赖未来帧，推理可直接去掉；教师的 AIT 可看完整片段

Role-aware 3D RoPE

视频： (t, h, w)； AIT： (t_n, H_v, W_v) 位于网格外； SIT： (t_n, 网格中心)

t_n = n · s_a / (s_v · d_t)： 动作位置映射到视频 latent 时间轴

教师看完整未来片段，学生只看首帧：推理时零额外开销

特权教师（冻结）

Video DiT + AIT； AIT 可注意完整未来片段 $f_0 \ldots f_h$

未来帧真值 latent 在 $\tau_v = 200$ 加噪，首帧保持干净

逐层 AIT 特征对齐（L2）

Video DiT 第 8 / 16 / 24 / 30 块

OSWAM 学生

AIT 只看首帧 + SIT + AIT； SIT 经注意力间接受监督

推理时丢弃教师 → 首帧接口不变，无额外计算

为什么要蒸馏

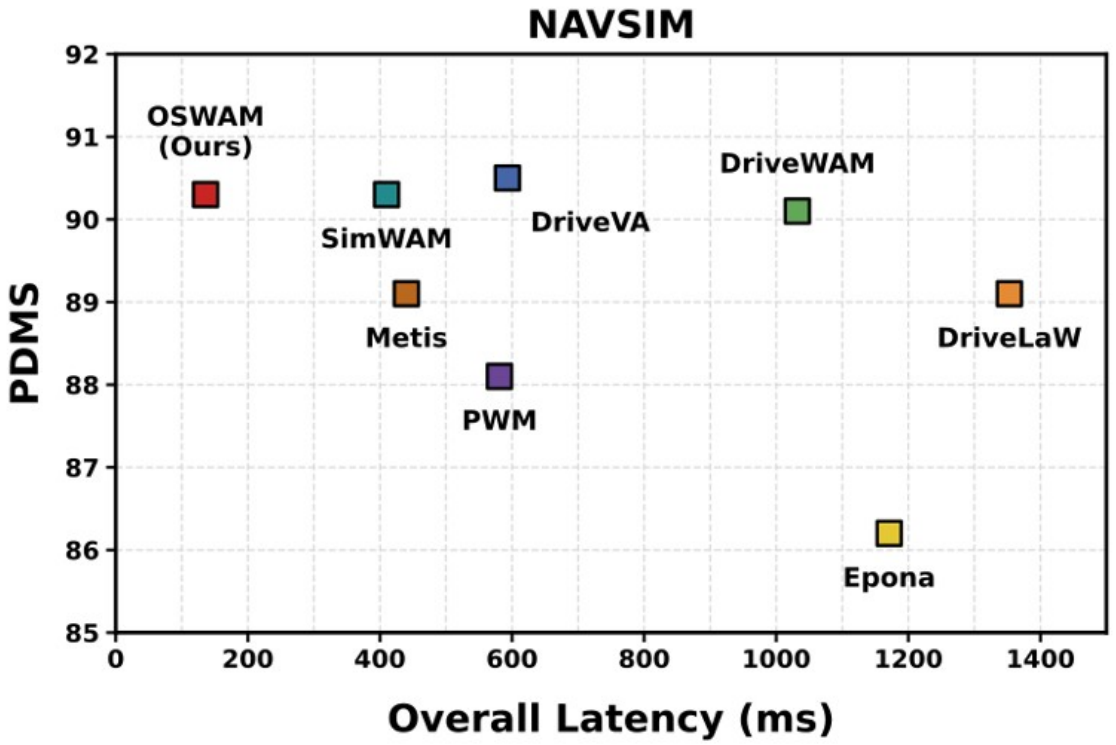
只用首帧时，长时程信息只能隐式压缩进首帧表征；把“看过未来”的教师表征迁移给 AIT，可以增强面向未来的动作生成。

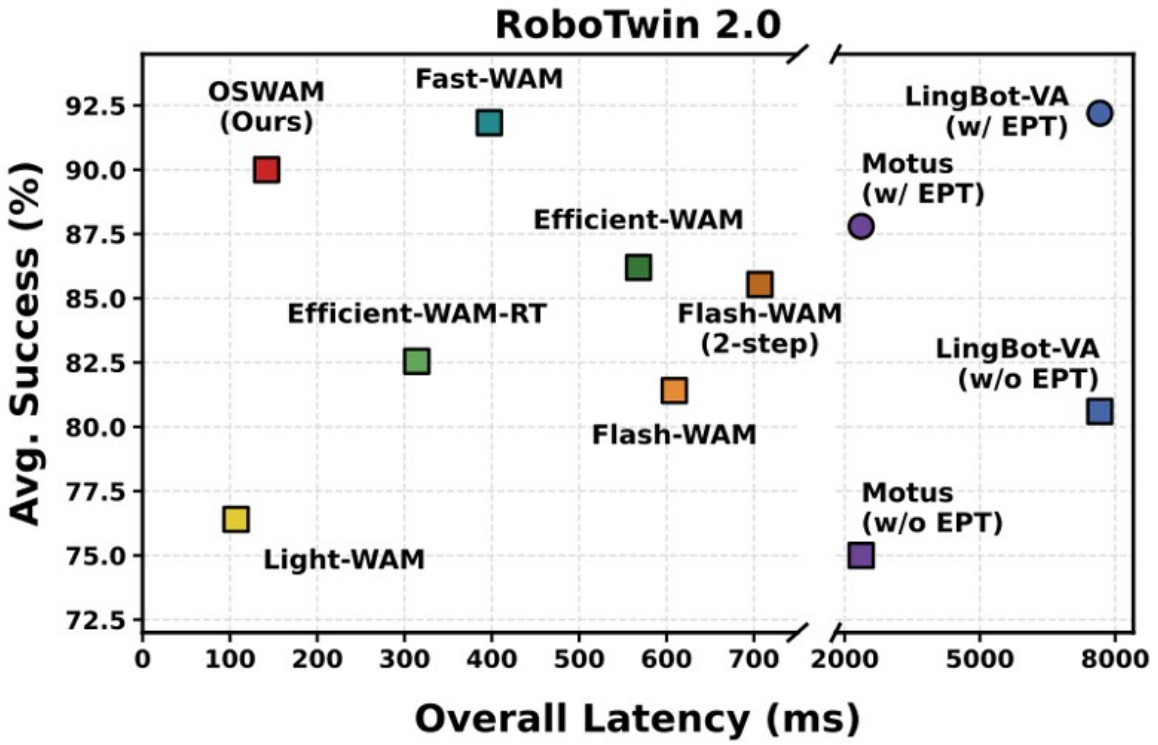
$$L_{dist} = |J|^{-1} \cdot \Sigma_j \| h_j^{stu} - sg(h_j^{tea}) \|^2$$

- 教师与学生共享视频噪声和动作噪声样本
- $\lambda_{dist} = 10$；训练 500 步后开始蒸馏，再线性升温 500 步
- 教师的训练设置与 OSWAM 相同，只有 token 组和掩码不同

消融增益（Table 4）： NAVSIM +0.15 PDMS； RoboTwin Clean +2.0 / Rand. +1.5

效率 - 性能平面上， OSWAM 在两个基准中都位于左上角





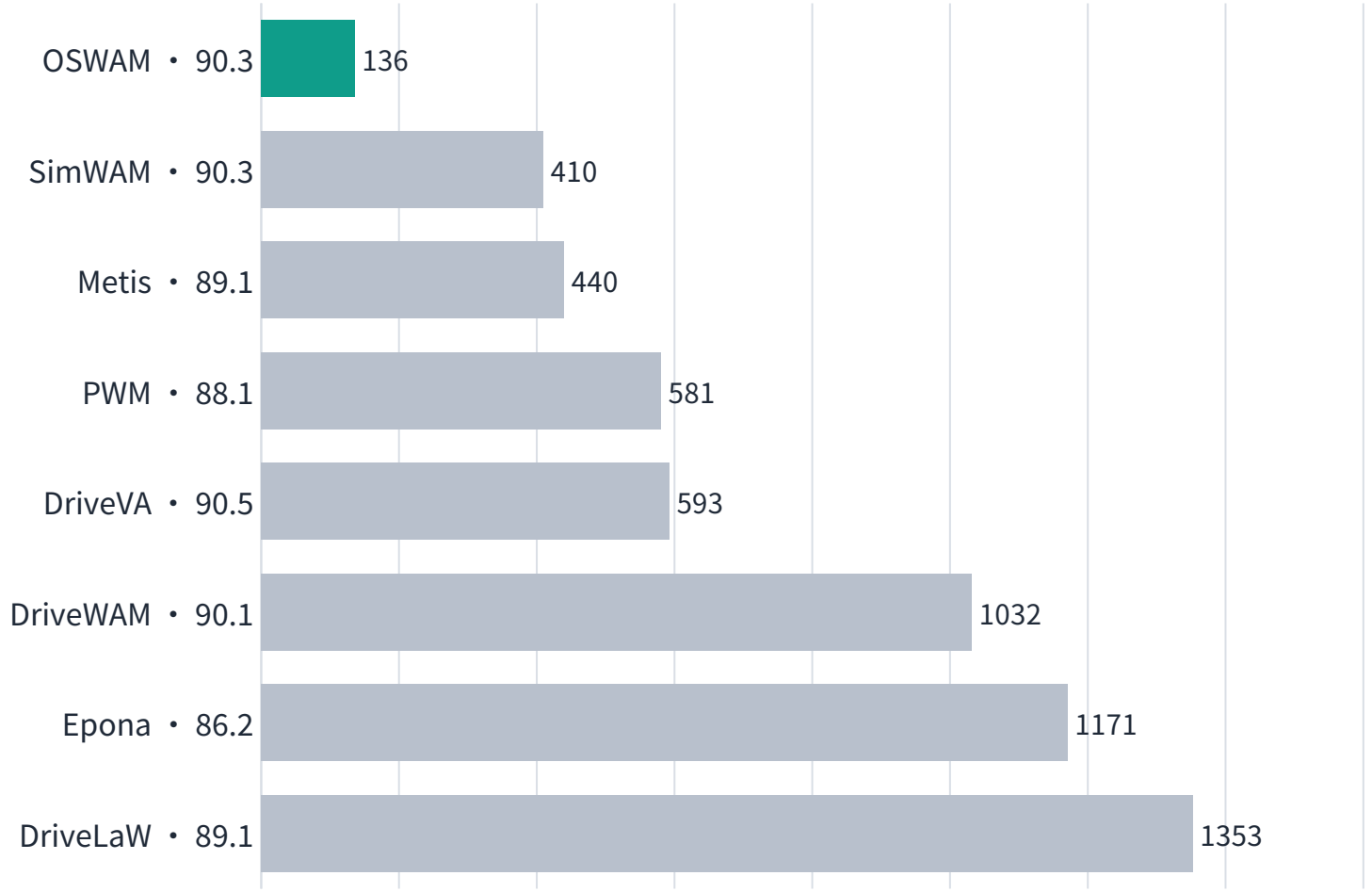
NAVSIM： 136 ms / 90.3 （SimWAM 410 ms / 90.3）

RoboTwin： 142 ms / 90.0% （Fast-WAM 397 ms / 91.83%）

出处： Fig. 1 （延迟为 NVIDIA H20 实测）

NAVSIM： PDMS 90.3 与 SimWAM 持平，延迟 410 → 136 ms

整体延迟（ms，越低越好）；标签：方法 · PDMS



3.01×

vs SimWAM （10 步）

PDMS 同为 90.3

3.24× +1.2

vs Metis （10 步）

PDMS 90.3 vs 89.1

4.36× −0.2

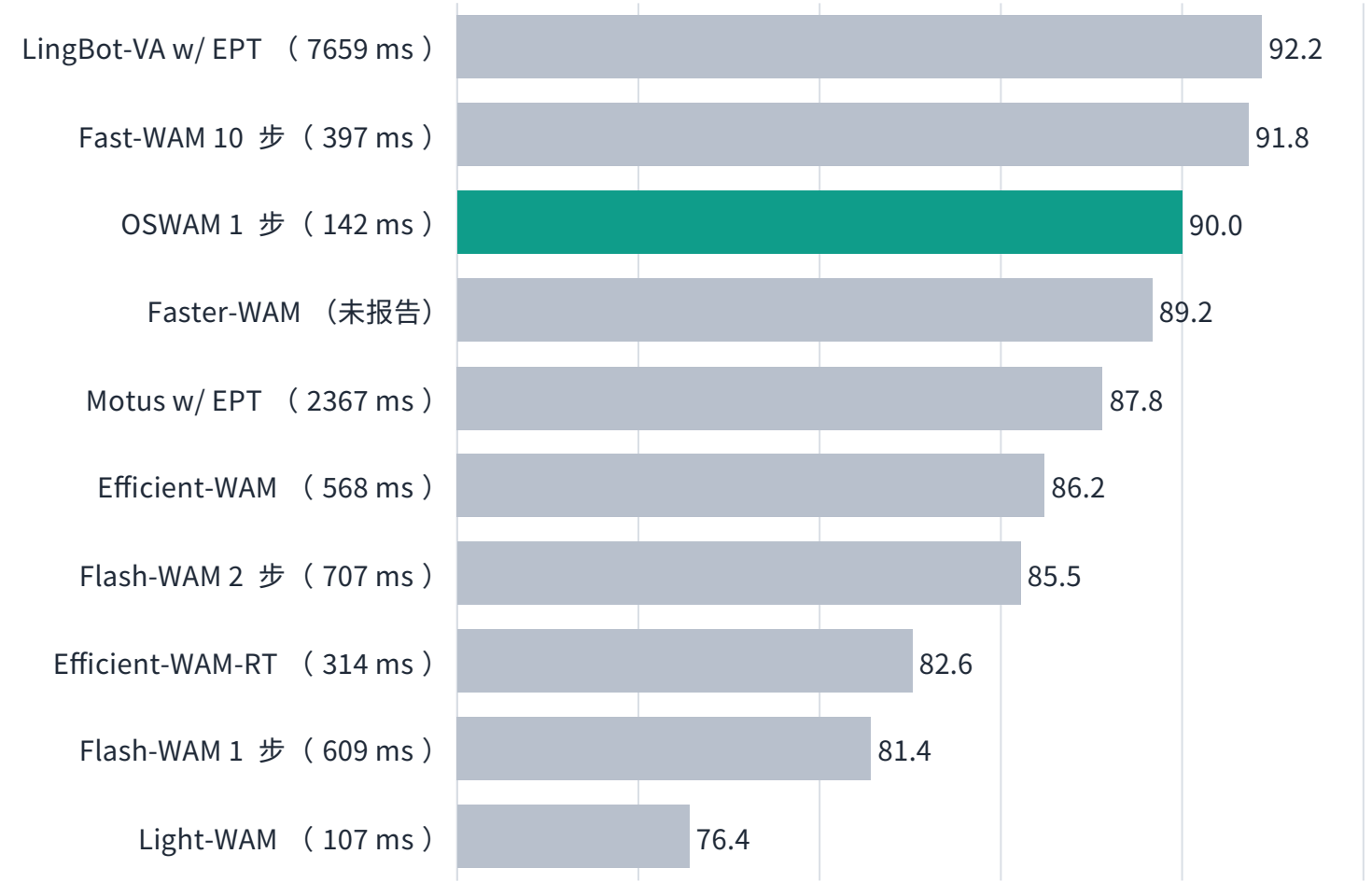
vs DriveVA （2 视频步 +2 动作步）

PDMS 90.3 vs 90.5

出处： Table 1 （NAVSIM navtest， H20； Metis、 DriveVA 延迟为作者复现）

RoboTwin 2.0：仅比 10 步 Fast-WAM 低 1.83 个点，提速 2.80 倍

平均成功率（%，越高越好）；括号内为延迟； EPT = 具身预训练



−1.83 / 2.80×

vs Fast-WAM （10 步）

90.00% vs 91.83%

+8.59 / 4.29×

vs Flash-WAM （1 步）

90.00% vs 81.41%

+13.6 / +35 ms

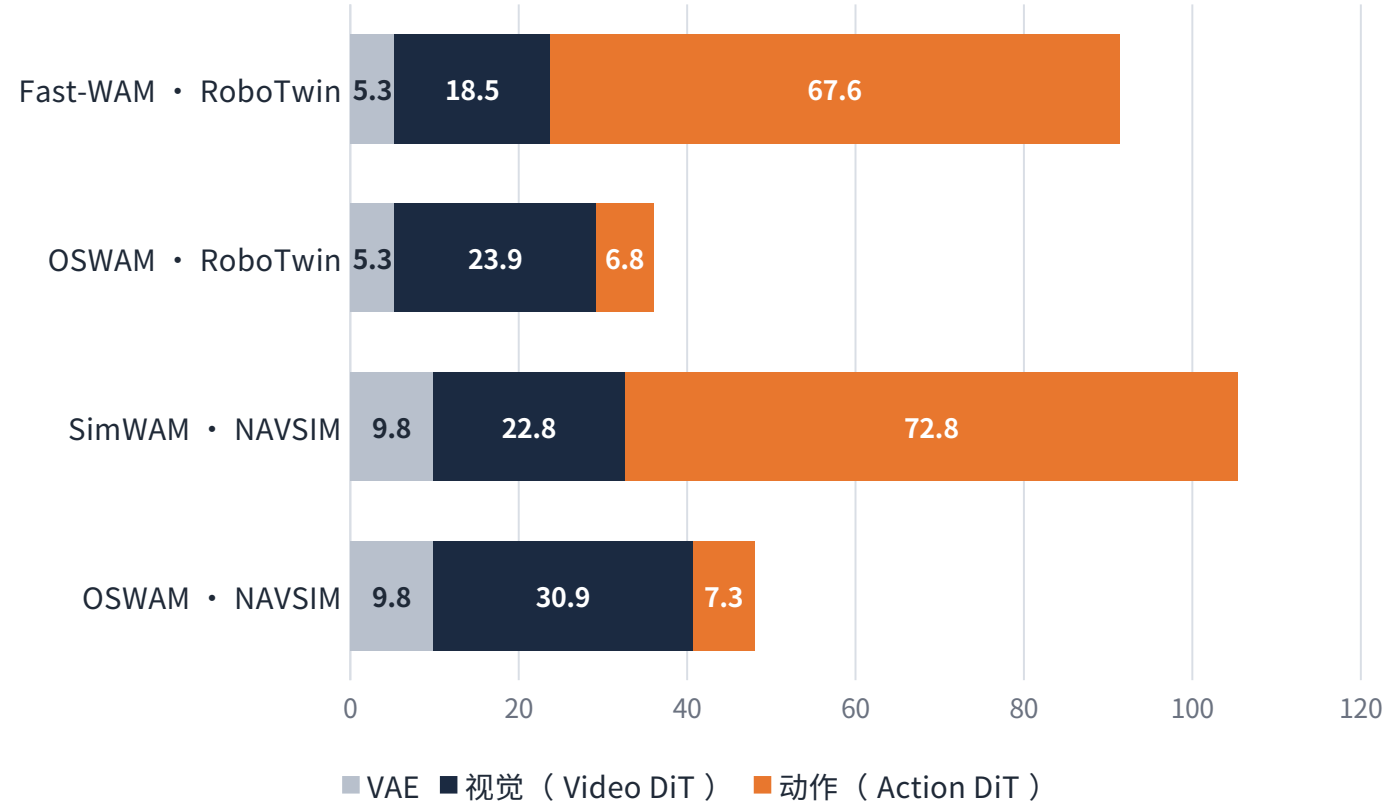
vs Light-WAM （单次前向）

90.00% vs 76.40%

出处： Table 2 （50 个任务， Clean / Randomized 平均； H20 ）

动作 GPU 时间约降为 1/10，想象 token 只多花约 5–8 ms

GPU 分项时间（ms）



模型	整体延迟	编译后
Fast-WAM	323.67	73.64
OSWAM	**95.35**	**40.81**
SimWAM	330.8	90.53
OSWAM	**99.70**	**52.64**

上两行 RoboTwin，下两行 NAVSIM

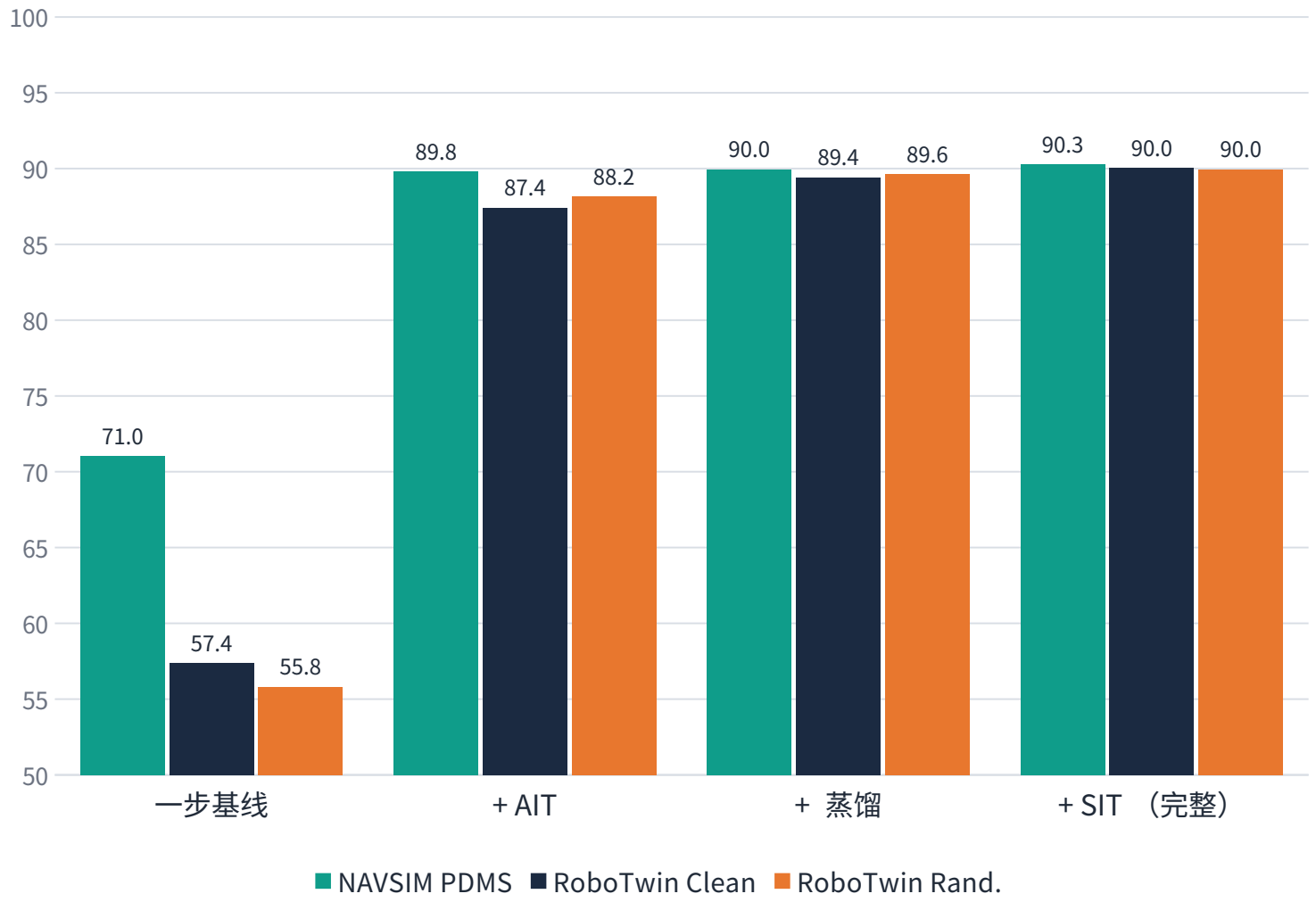
3.39×
RoboTwin
整体延迟

3.32×
NAVSIM
整体延迟

GPU 分项合计约 36 ms，整体 95 ms 中其余为 CPU / 调度开销；torch.compile + CUDA Graph 编译后降至 40.81 ms。峰值显存 12.70 → 12.75 GB

出处：Table 3（RTX 5090，语言上下文已缓存，单位 ms）

AIT 贡献了几乎全部提升；蒸馏和 SIT 是锦上添花



+18.8 / +30

+ AIT （结构化起点）

NAVSIM PDMS / RoboTwin Clean； 延迟约 +27 ms

+0.15 / +2.0

+ 特权蒸馏

RoboTwin Rand. +1.5； 推理零开销

+0.32 / +0.6

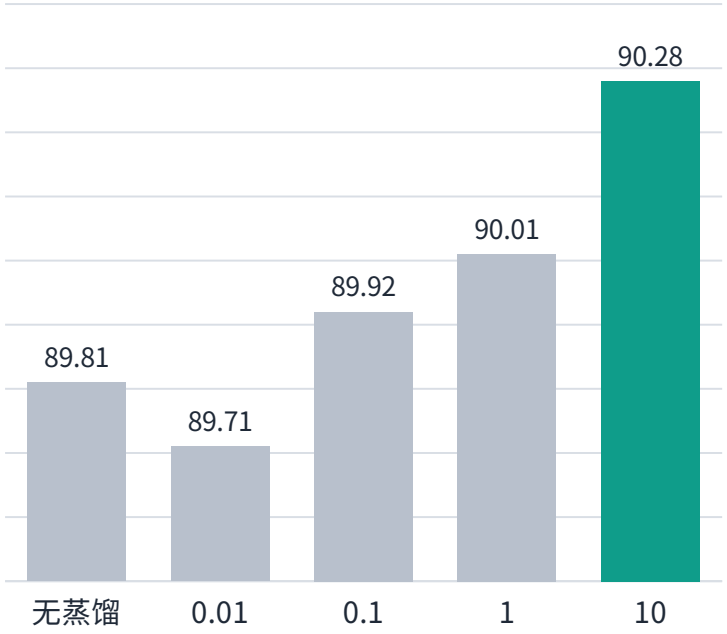
+ SIT

RoboTwin Rand. +0.3； 延迟 +0.2–2.9 ms

出处： Table 4 （基线 = 一步 Fast-WAM； 延迟为 H20 ）

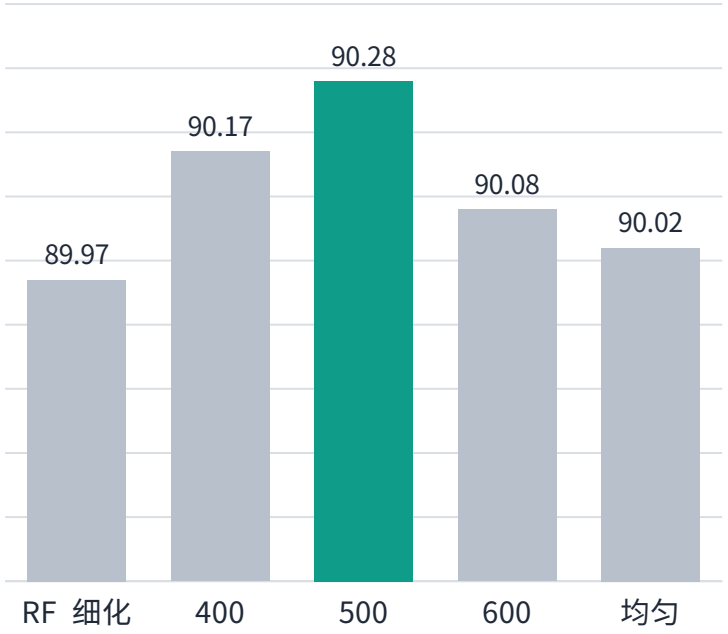
重加噪取中间水平最好；教师未来帧几乎全是噪声时效果也基本不变

蒸馏权重 λ_dist



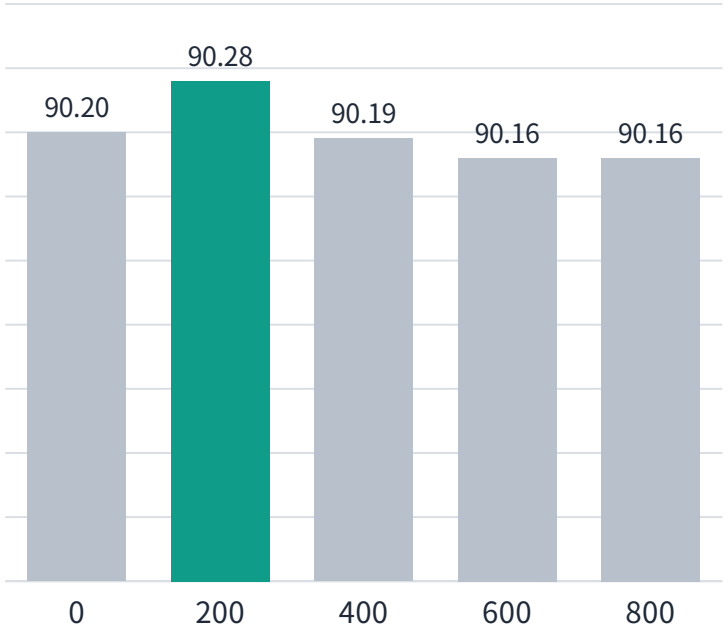
权重足够大才有效；λ = 0.01 反而略低于基线

重加噪水平 τ_r



中间噪声（σ = 0.5）最好，固定值优于均匀采样

教师视频时间步 τ_v



800（≈纯噪声）仍有 90.16 ≈ 干净的 90.20

注：各设置间差异多在 0.1–0.3 PDMS，论文未报告多随机种子的方差。

NAVSIM v2 上 EPDMS 最高； nuScenes 开环 L2 与碰撞率居前

NAVSIM v2 （EPDMS ↑）

方法	动作步数	EPDMS
Human Agent	–	90.3
DiffusionDrive	–	84.5
DriveVLA-W0	–	86.1
Epona	100	85.1
DriveLaW	5	88.6
Metis	10	89.5
SimWAM	10	90.2
OSWAM	**1**	**90.3**

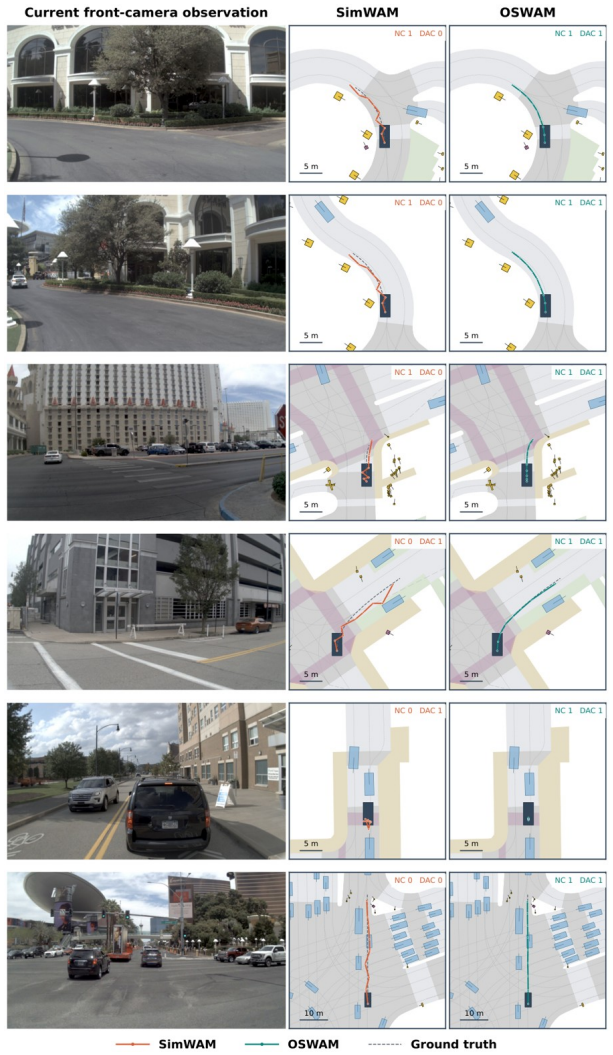
nuScenes 开环规划（均值，↓）

方法	辅助监督	L2 (m)	碰撞率（%）
UniAD	Map/Box/Motion…	1.03	0.31
VAD-Base	Map/Box/Motion	1.22	0.53
GenAD	Map/Box/Motion	0.91	0.43
Epona	无	1.25	0.36
DriveLaW	无	1.15	0.24
PWM	Text	0.41	0.04
OSWAM	**无**	**0.41**	**0.03**

读法： NAVSIM v2 上一步 OSWAM 小幅领先十步方法； nuScenes 为开环评测，可作参考，但不宜作为规划能力的强证据。

出处： Table A.1 （NAVSIM-v2 navtest）、 Table A.2 （nuScenes 开环）

同样只去噪一步： SimWAM 轨迹抖动、越界， OSWAM 平滑合规



怎么看这张图

- 每行一个场景：前视相机 | 一步 SimWAM | OSWAM
- 红线 = SimWAM ， 青线 = OSWAM ， 灰线 = 真值
- 右上角标注 NC （无责碰撞） / DAC （可行驶区域合规）

68.9 → 90.3

一步 PDMS： SimWAM → OSWAM

Table A.4

在转弯、复杂交互场景中，一步 SimWAM 越界或碰撞；
OSWAM 同样只去噪一步，轨迹平滑， NC 与 DAC 均满足。

出处： Fig. B.1 （ NAVSIM v1 ，一步 SimWAM vs OSWAM ）

PiPER 真机：成功率略高于 Fast-WAM，完成时间缩短 22–34%

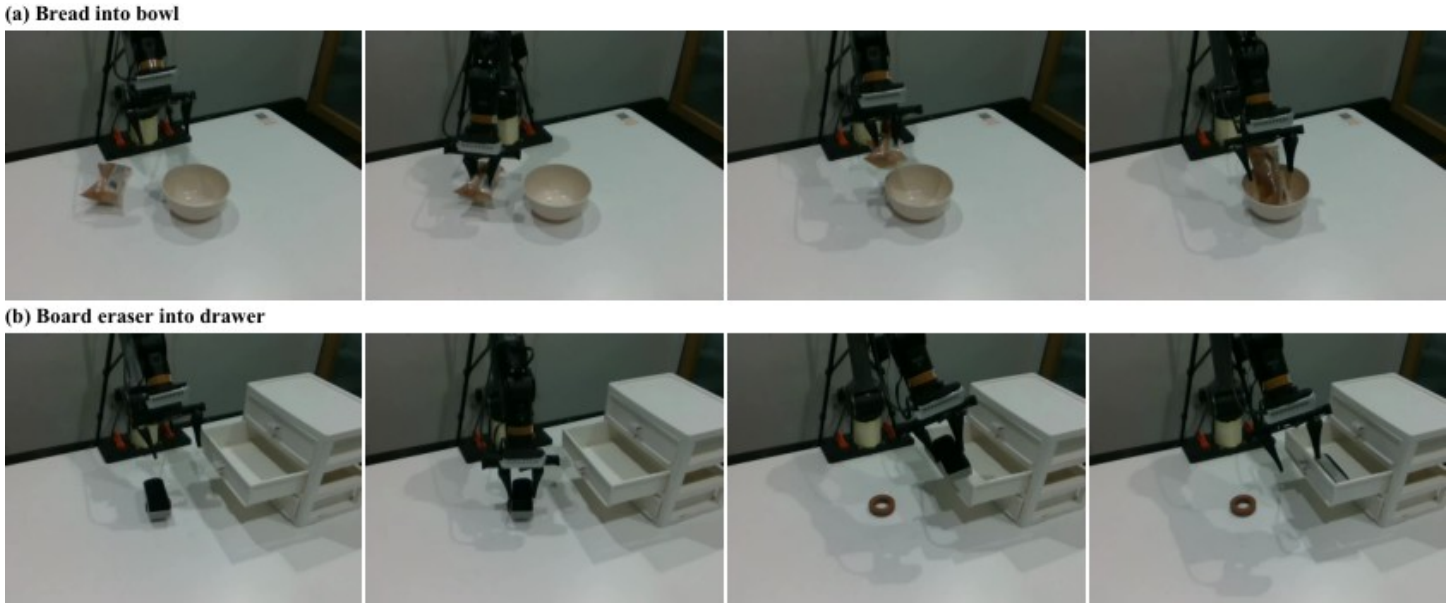


Fig. 5： (a) 面包放入碗中； (b) 板擦放入打开的抽屉（需避开抽屉口与柜体）

任务	Fast-WAM	OSWAM
面包 → 碗 成功率	80%	85%
板擦 → 抽屉 成功率	75%	80%

−34.2%

面包任务

24.6 → 16.2 s

−21.8%

板擦任务

17.6 → 13.8 s

注意

每个任务仅 20 次， 5 个百分点 = 1 次成功，统计上不显著；完成时间（成功回合，按视频估计）的缩短更能体现低延迟的价值。

出处： Table 8 、 Fig. 5 （每个任务 50 条示教、测 20 次）

值得追问的六个点

1. “一步”的真实含义

实为“Video DiT 回归粗动作 + Action DiT 一次残差修正”；共享 ε，接近确定性映射，未讨论多模态动作分布

2. 蒸馏增益偏小

NAVSIM 仅 +0.15 PDMS；教师未来帧加噪到 τ = 800 仍有 90.16 ≈ 90.20 → “特权未来信息”存疑；无方差报告

3. 并未完全追平

RoboTwin 比 Fast-WAM 低 1.83 个点；逐任务仅 22/50（Clean）、15/50（Rand.）持平或超过

4. 延迟口径不一

主表用 H20，效率表用 RTX 5090；部分基线为复现；40 ms 为编译后数字，不宜与他人未编译数字直接比

5. 真机规模小

每个任务仅 20 次试验，成功率差 5 个百分点 = 1 次；完成时间缩短更有说服力

6. 先验只作用于初始化

动作 token 不直接注意 AIT / SIT；把场景上下文与特权监督用到 Action DiT 本身尚未探索（作者亦承认）

总结

一个简单有效的思路：让世界模型主干顺带给出动作的起点

问题	直接策略式 WAM 已去掉视频生成，剩下的瓶颈是约 10 步动作去噪；根因是纯噪声起点缺少动作结构
方法	AIT / SIT 在 Video DiT 一次前向中产出结构化先验 → 重加噪到 $\sigma = 0.5$ → Action DiT 一步修正；特权教师蒸馏未来信息
效果	NAVSIM PDMS 90.3 （持平）、 RoboTwin 90.0% （− 1.83 ）；约 3× 提速，编译后 40–53 ms ，可支持 10 Hz 以上控制

启发： 主要贡献来自“结构化起点”（ AIT ）；这一思路可迁移到其他扩散 / flow 策略的实时部署。蒸馏与 SIT 的增益较小，其机制仍待更严格验证。